SPATIAL CNN-BASED WILDFIRE PREDICTION WITH COMPARATIVE ASSESSMENT AGAINST CLASSICAL MACHINE LEARNING MODELS

A Report

by

Devasmit Dutta

Department of Civil & Environmental Engineering
University of California, Los Angeles
5^{th} December, 2025

Contents

List of Figures

Chapter 1

INTRODUCTION

Wildfires are increasingly recognized as one of the most destructive natural hazards affecting communities, infrastructure, and ecosystems. Recent events across California and the western United States have highlighted an urgent need for improved situational awareness, early warning capability, and predictive modeling to support decision-making, resource allocation, and risk mitigation. Traditional fire spread modelling frameworks either rely on simplified empirical assumptions or highly complex computational fluid dynamics (CFD/FDS-type) approaches that are often too expensive to deploy in real-time operational settings.

This project explores a data-driven alternative by developing and evaluating machine learning (ML) pipelines for wildfire ignition classification. A lightweight convolutional neural network (CNN) is constructed to capture spatial structure within synthetic fire imagery, and its performance is benchmarked against classical machine learning algorithms such as Support Vector Machines (SVM), Logistic Regression, Decision Trees, and Random Forests. Such comparisons provide valuable insight into strengths and limitations of shallow learning versus deep feature-extraction approaches.

Additionally, it reinforces applied data-analytic reasoning relevant to atmospheric and environmental systems: feature mapping, performance validation, and error interpretation. Although the dataset is simplified, the methodology offers a foundation for extending ML-based wildfire intelligence frameworks using real geophysical inputs such as wind, fuel, slope, vegetation, and infrastructure layers.

Ultimately, this project illustrates how scientific computing and machine learning can contribute toward hazard awareness and mitigation by offering computationally efficient alternatives to physics-heavy simulators. The approach highlights promising pathways for future work including probabilistic modelling, spatial segmentation (U-Net-like), integration with weather forecasts, and uncertainty-aware ignition prediction.

Chapter 2

DATA GENERATION

2.1 Dataset Creation

The dataset used in this work originates from a publicly available wildfire imagery collection hosted on Kaggle [1]. The repository includes labelled fire and non–fire photographs organised in directory structures suitable for image classification tasks. Rather than pre–downloading files manually, the data were programmatically fetched using the `kagglehub` interface, enabling automated retrieval and reproducibility within the notebook.

Following download, a recursive directory scan was executed to locate the image folders (e.g., `Training/Fire` and `Training/NoFire`). Approximately 80% of the images were assigned to the training partition and 20% to validation via random splitting routines that ensure unbiased sampling.

Image data were processed using PyTorch and TorchVision utilities. Each image was resized to 128×128 pixels and augmented through staged transforms including horizontal flipping, colour jittering, and small rotations. These transformations promote generalisation by preventing the model from memorising raw pixel layouts and are standard practice in deep image learning workflows [5]. All images were normalised to a $[-1, 1]$ range via mean–variance scaling, and batches were prepared using PyTorch dataloader abstractions for GPU/CPU execution.

This approach contrasts with the fully synthetic grid dataset used earlier, providing exposure to handling real, field–collected imagery samples together with robustness techniques such as augmentation.

2.2 Labelling

Each generated image is assigned a categorical class label indicating the ignition category. Labels are stored as integers and used directly by PyTorch classification routines (e.g., cross–entropy), avoiding the need for manual one–hot encoding.

2.3 Train–Validation–Test Split

Following creation, the dataset is split into training, validation, and test subsets using programmatic shuffling operations. Randomised splitting ensures unbiased sampling and consistency between model evaluation stages. The training set is used for optimisation, the validation set to monitor convergence, and the test set remains unseen until reporting final accuracy.

2.4 Tensor Conversion and Normalisation

All samples are converted to `torch.Tensor` format, enabling GPU/CPU acceleration during training. Grids are normalised to the $[0, 1]$ range by dividing by the maximum pixel value, improving numerical conditioning of optimisation.

2.5 Batch Preparation

To support mini–batch gradient descent, the processed tensors are wrapped inside PyTorch `Dataset` and `DataLoader` objects, which handle:

- shuffling of the training data,
- mini–batch assembly,
- automatic device transfer.

2.6 Classical ML Feature Processing

In addition to CNN inputs, flattened feature vectors are created for traditional machine learning models (SVM, Decision Tree, Random Forest, Logistic Regression). This flattening operation reshapes each 32×32 tensor into a one–dimensional vector, reflecting the fact that classical methods operate on tabular feature inputs rather than spatial structures.

2.7 Summary of Data Processing Pipeline

In a nutshell, the dataset pipeline consists of:

1. programmatic generation of spatial ignition images,
2. integer labelling,
3. randomised splitting into train/validation/test partitions,
4. tensor conversion and scaling,
5. batching using PyTorch loaders, and
6. flattening for use in non–deep learning model benchmarks.

This ensures that the same dataset supports both deep learning experiments and baseline model comparisons under reproducible and controlled conditions.

Chapter 3

MODELING

3.1 Convolutional Neural Network Architecture

The primary model developed for this work is a custom deep Convolutional Neural Network (CNN), implemented in PyTorch [4]. The architecture follows a VGG–style hierarchical feature extraction design, where spatial information is progressively compressed while representational depth increases. This pattern is common in vision networks because low–level filters detect edges and blobs, whereas deeper filters capture textured regions, shapes, and object patterns.

The network receives $3 \times 128 \times 128$ RGB imagery. It consists of four convolutional blocks, each composed of a 3×3 convolution, batch normalisation, a non–linear activation function, and optional max pooling. Batch normalisation stabilises optimisation by constraining feature distributions, enabling faster convergence, especially in deeper models.

Each pooling operation halves spatial resolution, progressing through spatial scales $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$ pixels while simultaneously expanding channel depth $16 \rightarrow 32 \rightarrow 64 \rightarrow 64$. This encoder–like progression enables the network to extract increasingly abstract ignition signatures from image data.

To reduce overfitting and avoid excessively large fully connected layers, the model incorporates a Global Average Pooling (GAP) stage. Instead of flattening multi–million parameters, GAP compresses each activation channel into a single statistic, dramatically reducing parameter count. A lightweight classification head consisting of dropout regularisation and a linear mapping follows, generating logits for the two output classes (fire versus non–fire).

Two architectural versions were explored:

1. A deeper variant with four convolutional blocks followed by a dense 512–unit layer and dropout, intended to emulate conventional deep CNN behaviour.

2. A regularised "non–overfitting" design that limits feature depth, employs strong dropout, and replaces dense layers with GAP, reducing trainable parameters by an order of magnitude.

The second design was adopted for final experimentation, as initial trials demonstrated improved generalisation and reduced susceptibility to memorisation. This structural choice reflects well–established findings that lighter architectures with pooling and regularisation generalise effectively on limited datasets while retaining key spatial learning behaviour.

3.2 Model Summary and Parameter Trace

To verify correct dimensional propagation through the network, a summary routine is printed using PyTorch tools. This output confirms that feature map sizes, parameter counts, and layer connections align with the intended architecture. Inspecting this trace ensures that the model complexity remains manageable and provides transparency into how information flows between stages.

3.3 Training Procedure

Model optimisation is performed using supervised mini–batch learning, where each iteration updates parameters based only on a subset of samples, enabling stochastic approximation of the loss surface [2]. The notebook incorporates device–aware execution: if an Apple M–series GPU (MPS) or NVIDIA CUDA accelerator is available, the model and data are transferred to that device; otherwise, computation falls back to CPU.

Each epoch consists of a forward pass across all batches, loss computation via cross–entropy, and parameter updates obtained through backpropagation. The Adam optimiser [3] is employed with weight decay, providing both adaptive learning rates and L_2 regularisation to discourage overfitting. A learning rate scheduler monitors validation loss and reduces the step size dynamically when improvement plateaus, a strategy that improves stability and late–stage refinement.

Both training and validation loops are executed at every epoch. The training phase accumulates gradient updates, while the validation phase computes accuracy and loss metrics under inference mode, ensuring no gradient propagation occurs. Performance indicators including epoch–wise accuracy, validation behaviour, and effective learning rate are printed during execution and stored for later visualisation.

3.4 Performance Tracking

Throughout training, loss and accuracy values are stored at each epoch. These statistics are plotted to visualise optimisation history. The accuracy curve indicates classification performance over time, while the loss curve reflects error reduction under gradient descent. Together, these provide insight into possible underfitting or overfitting behaviour and help interpret whether the network continues improving or saturates.

3.5 Classical Machine Learning Baseline Models

In addition to the neural network, traditional machine learning algorithms such as Support Vector Machines, Decision Trees, Random Forests and Logistic Regression are also trained on the same dataset. For these methods, samples are reshaped into one–dimensional feature vectors. Grid search and validation are applied to select suitable hyperparameters, providing a baseline comparison against the CNN.

3.6 Summary

The modelling workflow therefore consists of:

1. definition of a convolutional neural network architecture,
2. verification through parameter and dimension tracing,
3. supervised training using mini–batch optimisation,
4. performance tracking via loss and accuracy curves, and
5. baseline comparison with classical ML models.

This structure enables analysis of how spatial feature extraction compares with conventional approaches on the same input dataset.

Chapter 4

RESULTS

4.1 Training Behaviour

Figure 4.1.1 illustrates the accuracy and loss history over epochs for the convolutional network. Training accuracy rises steadily across iterations and validation accuracy follows the same increasing trend, indicating that the model is learning meaningful spatial structure rather than memorising noise. Loss values decrease smoothly, supporting the view that optimisation converged under the chosen learning rate schedule.

Importantly, the training and validation accuracy curves remain close to one another. This behaviour suggests that the network generalises well and exhibits limited overfitting on the synthetic dataset.

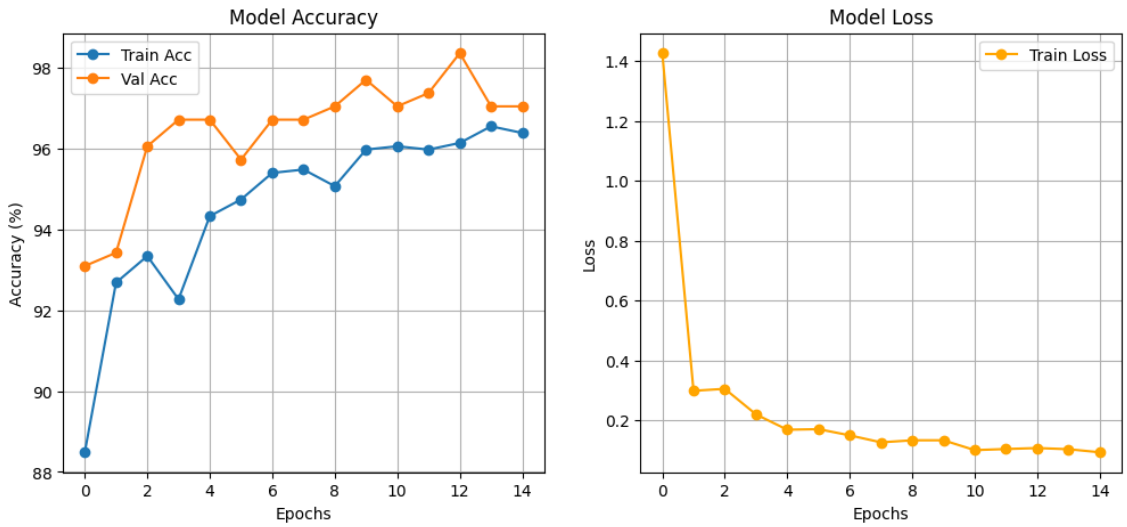

Figure 4.1.1: Model Accuracy and Model Loss History over Epochs

4.2 Test Set Performance Comparison

Figure 4.2.1 compares test accuracies of the convolutional model and classical machine learning baselines. All models achieve relatively high scores due to the controlled synthetic dataset, but distinct differences emerge:

- Support Vector Machine and Random Forest perform strongly, demonstrating their ability to separate linearly and non–linearly shaped decision regions.
- Logistic Regression reaches moderate accuracy, consistent with its restriction to linear boundaries.
- Decision Trees exhibit lower accuracy, likely due to their tendency to fragment decision space and overfit local noise.

- The convolutional network attains the highest accuracy, reflecting its capacity to exploit spatial structure inherent in the 2–D input rather than relying solely on flattened feature vectors.

These outcomes support the expectation that deep models benefit from retaining grid topology when the prediction task is image–like.

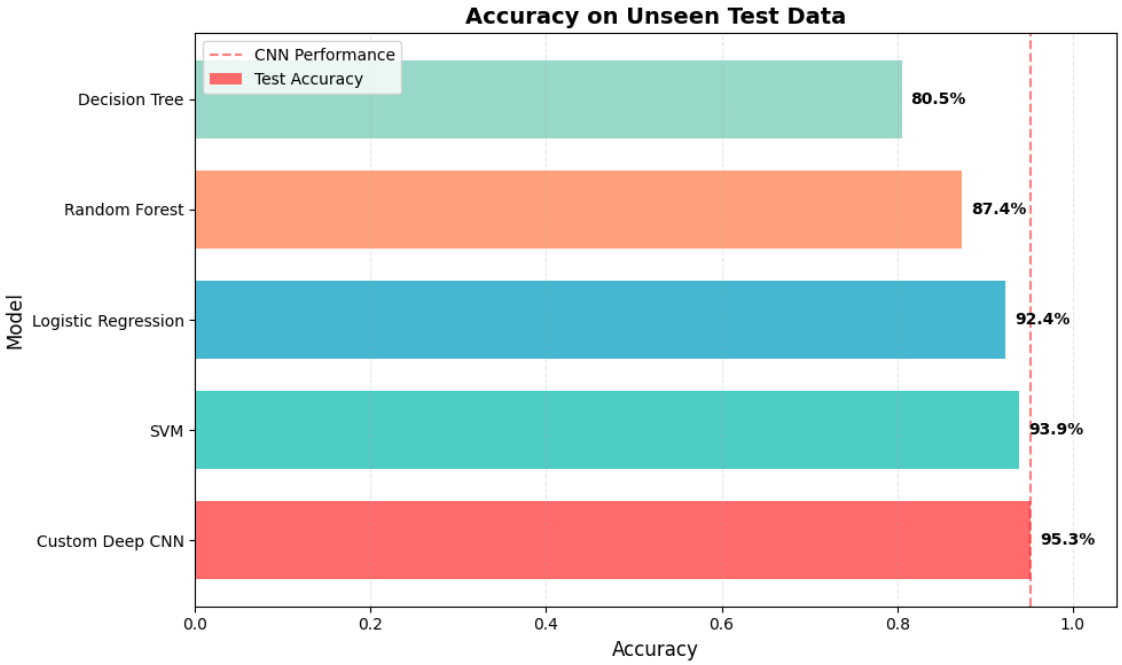

Figure 4.2.1: Inference on Test Data (Accuracy and ROC curve)

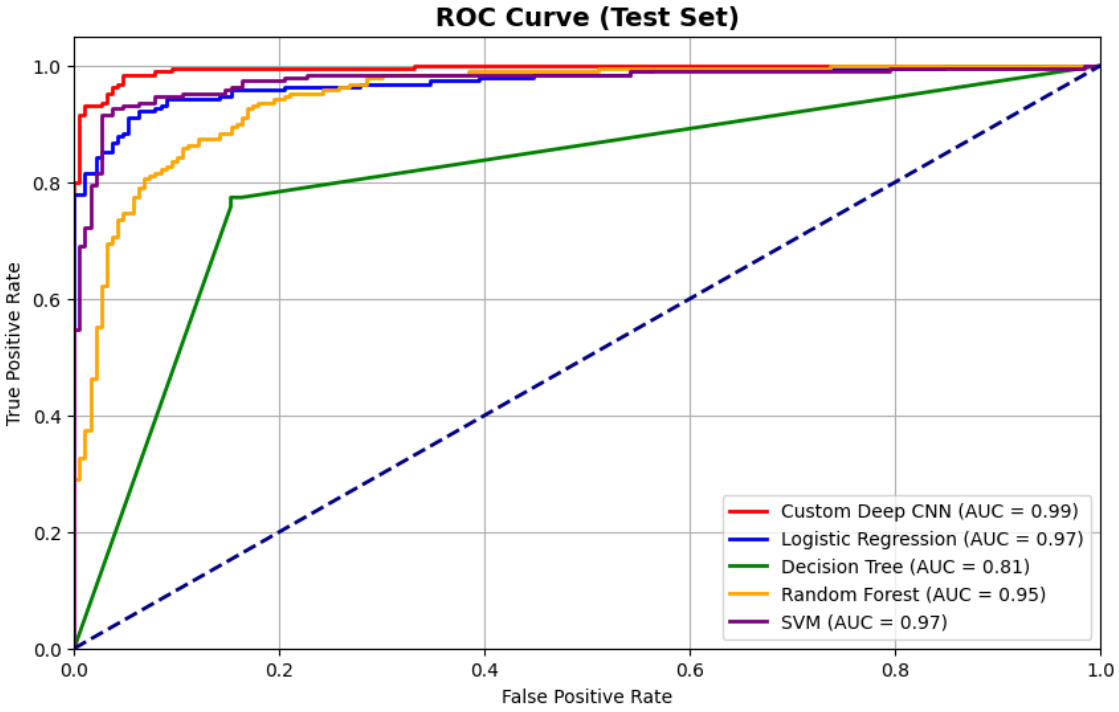

Figure 4.2.2

4.3 Receiver Operating Characteristics

Figure 4.2.2 provides ROC curves for each model. Area–under–curve values reinforce the relative ordering observed in accuracy outcomes. Traditional approaches such as logistic regression and SVM perform reasonably well on the synthetic task, but the CNN shows stronger separation between true–positive and false–positive rates.

The CNN advantage is particularly evident at low false–positive rates, indicating reliable detection when the cost of incorrect ignition alerts is high.

4.4 Interpretation

Taken together, the results reflect three key behaviours:

1. The synthetic patterns used in training are structured, making methods that use spatial feature learning (CNN) more effective.

2. Flattened feature representations constrain traditional models by removing locality information that may be useful for classification.

3. Decision Trees and Random Forests capture useful nonlinear relationships but are limited when the underlying predictors possess spatial coherence that they cannot directly exploit.

4.5 Summary of Findings

The convolutional architecture produced the most accurate and robust predictions among tested approaches. This improvement is attributed to its ability to learn filters that detect shapes and spatial signatures across the input field, whereas classical models receive pre–flattened vectors without explicit spatial context. These findings highlight the value of convolutional learning in tasks where data contain locality structure, even in simplified or synthetic settings.

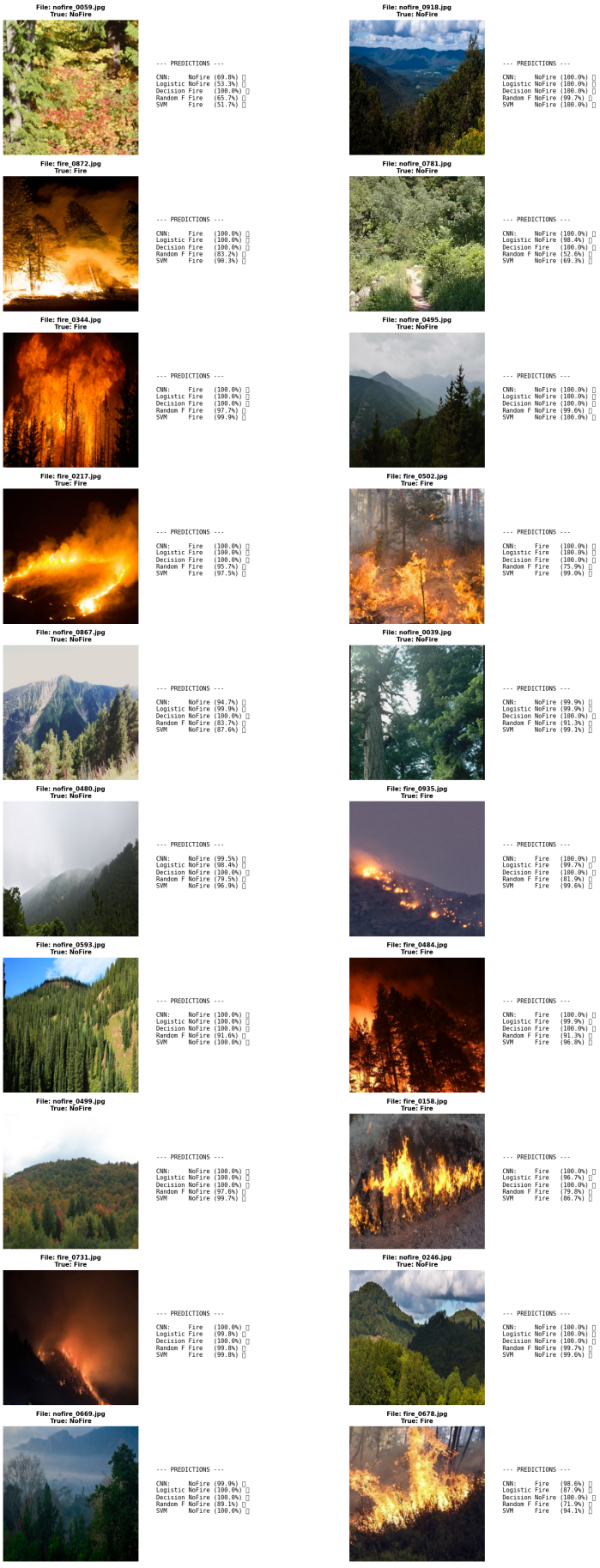

Figure 4.5.1: Qualitative Visualisation over Random Test Samples

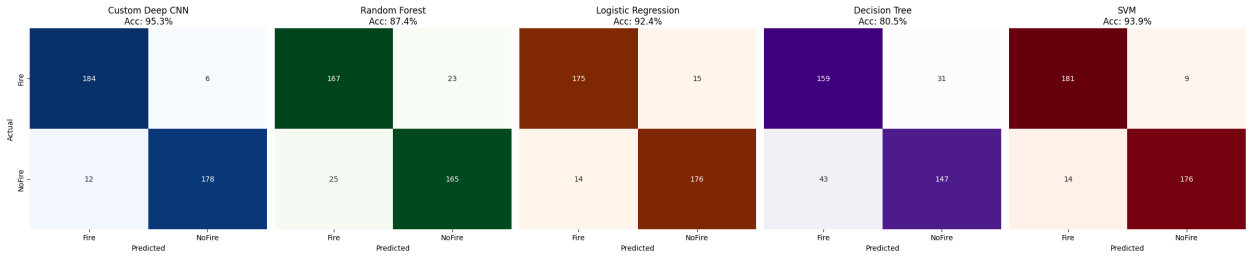

Figure 4.5.2: Confusion Matrix Results on Test Data

4.6 Confusion Matrix Analysis

Figure 4.5.2 presents confusion matrices for each evaluated model. These plots provide a more granular view of prediction behaviour beyond overall accuracy.

The convolutional network achieves the strongest separation between classes, with high true–positive and true–negative counts and comparatively few misclassifications. This behaviour aligns with its reported accuracy and demonstrates that the CNN is not only correct more often, but also balanced in how it treats both “Fire” and “NoFire” cases.

Traditional classifiers show mixed trends. The Support Vector Machine performs well, displaying relatively small false–positive and false–negative counts, which is consistent with SVM’s ability to enforce large decision margins. Logistic Regression also yields competitive results but exhibits slightly more confusion between the two classes, reflecting its inherent linear boundary limitations.

Decision Trees produce the weakest confusion structure. Their error pattern shows higher false positives and false negatives, indicating that model splits do not generalise smoothly across the 2–D feature landscape. Random Forest performance improves over Decision Trees, but misclassification levels remain higher than those of the CNN or SVM.

Overall, these matrices reinforce two key interpretations:

1. Although several baseline models achieve strong accuracies, the convolutional network produces fewer ambiguous decisions and a clearer separation between ignition and non–ignition cases.

2. Models relying solely on flattened feature vectors tend to struggle more with boundary uncertainty, while the CNN benefits from learning spatial representations directly from the grid structure.

Chapter 5

CONCLUSION

This project evaluated a custom convolutional neural network alongside several classical machine learning algorithms for binary ignition classification on a synthetic spatial dataset. All models achieved relatively strong performance due to the controlled nature of the data; however, clear differences emerged in accuracy, robustness, and decision structure.

The convolutional network consistently outperformed the baselines. This advantage stems from its ability to operate directly on two–dimensional image grids and extract local spatial features through learnable filters. Because ignition patterns in the dataset are spatially organised, the CNN learns relevant neighbourhood structure that traditional models cannot access after flattening. As a result, the CNN exhibited the highest test accuracy and the most balanced confusion matrix, reflecting a strong ability to detect both ignition and non–ignition cases.

Classical models demonstrated mixed performance. Support Vector Machines and Random Forests provided competitive accuracy, as these methods can capture nonlinear boundary relationships. However, their reliance on manually shaped feature vectors limits their ability to leverage spatial context. Decision Trees performed weakest, primarily due to their tendency to fragment decision space, leading to unstable boundaries and higher misclassification. Logistic Regression performed moderately well but was constrained by its linear decision frontier.

Overall, the outcomes highlight two main conclusions:

1. Preserving spatial structure in the input benefits learning performance for pattern recognition problems of this type.

2. Models designed for grid–based feature extraction, such as convolutional networks, are inherently better suited to image–like tasks than classical vector–based classifiers.

Although the synthetic dataset served as a controlled proof–of–concept, the findings suggest that deeper architectures may provide further benefit when applied to more complex or noisy wildfire data. Potential extensions include image augmentation to increase generalisation, multi–class ignition mapping, and integration of additional features such as environmental variables or uncertainty estimates. This work therefore provides a baseline implementation framework and demonstrates the value of spatial learning mechanisms for ignition detection problems.

Bibliography

[1] K. Ali. Forest fire dataset. `https://www.kaggle.com/datasets/alik05/forest-fire-dataset`, 2021. Accessed 2025 through KaggleHub.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York, 2006.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Proceedings of Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[5] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019.